

PCA: Principal Component Analysis

Analítica de Datos, Universidad de San Andrés

Si encuentran algún error en el documento o hay alguna duda, mandenme un mail a rodriguezf@udesa.edu.ar y lo revisamos.

1. El Problema de la Alta Dimensionalidad

Imaginá que estás trabajando con un dataset de vinos que tiene 13 propiedades químicas: acidez fija, acidez volátil, ácido cítrico, azúcar residual, cloruros, dióxido de azufre libre, dióxido de azufre total, densidad, pH, sulfatos, alcohol, etc. O peor: imaginá un dataset con cientos o miles de variables.

Esto genera varios problemas:

- **Imposible de visualizar:** no podés graficar 13 dimensiones en una pantalla 2D
- **Multicolinealidad:** muchas de estas propiedades están correlacionadas (por ejemplo, acidez con pH, alcohol con densidad)
- **Costo computacional alto:** entrenar modelos con muchas features es lento y consume mucha memoria
- **Curse of dimensionality:** en espacios de alta dimensión, los datos se vuelven “dispersos” y las distancias pierden significado
- **Overfitting:** más features = más riesgo de sobreajuste, especialmente si tenés pocas observaciones

La pregunta es: **¿Podemos reducir el número de features sin perder demasiada información?** Acá entra PCA.

2. ¿Qué es PCA?

PCA (*Principal Component Analysis* o Análisis de Componentes Principales) es una técnica de **reducción de dimensionalidad**. Su objetivo es transformar un conjunto de variables (posiblemente correlacionadas) en un conjunto

más pequeño de variables no correlacionadas llamadas **componentes principales**.

2.1. La Idea Intuitiva

Pensalo así: tenés un montón de features que están correlacionadas. Por ejemplo, en nuestro dataset de vinos:

- Acidez fija, acidez volátil, ácido cítrico, pH → todas relacionadas con la “composición ácida”
- Alcohol, densidad, azúcar residual → todas relacionadas con la “composición alcohólica”
- Sulfatos, dióxido de azufre libre/total → relacionadas con “conservantes”

En lugar de tener todas estas variables, PCA crea nuevas variables (componentes principales) que capturen la mayor parte de la variabilidad:

- **Componente Principal 1 (PC1)**: captura la dirección de máxima variabilidad en los datos (ej: “balance ácido general del vino”)
- **Componente Principal 2 (PC2)**: captura la segunda dirección de máxima variabilidad, **ortogonal a PC1** (ej: “contenido alcohólico y dulzor”)
- Y así sucesivamente...

La clave es que los primeros componentes principales suelen capturar la mayor parte de la información, entonces podés quedarte solo con ellos y descartar el resto.

2.2. ¿Por Qué Usar PCA?

Las principales motivaciones para usar PCA son:

1. **Visualización**: reducir a 2 o 3 dimensiones para poder graficar y explorar los datos visualmente

2. **Reducir costo computacional:** menos features = entrenamiento más rápido y menos memoria
3. **Preprocesamiento para otros modelos:** eliminar multicolinealidad antes de aplicar regresión lineal, por ejemplo
4. **Eliminación de ruido:** si las últimas componentes tienen poca varianza, probablemente sean ruido
5. **Compresión de datos:** almacenar datos de forma más eficiente

3. ¿Cómo Funciona PCA?

Vamos a explicar la intuición sin meternos demasiado en el álgebra lineal.

3.1. Paso a Paso

Paso 1: Estandarizar los datos

PCA es sensible a la escala de las variables. Si una variable tiene un rango mucho mayor que otra, va a dominar la varianza. Por eso, primero estandarizamos todas las features para que tengan media 0 y varianza 1.

$$z_i = \frac{x_i - \mu}{\sigma}$$

Paso 2: Calcular la matriz de covarianza

La matriz de covarianza nos dice cómo varían las features entre sí. Si dos features tienen covarianza alta (positiva o negativa), significa que están correlacionadas.

Paso 3: Calcular los eigenvectors y eigenvalues

Acá viene la magia del álgebra lineal. Los **eigenvectors** (autovectores) nos dan las **direcciones** de máxima varianza, y los **eigenvalues** (autovalores) nos dicen **cuánta varianza** hay en cada dirección.

- El eigenvector con el eigenvalue más grande define la primera componente principal (PC1)
- El segundo eigenvector define PC2, y así sucesivamente

- Los eigenvectors son ortogonales entre sí (perpendiculares), lo que garantiza que los componentes principales no estén correlacionados

Paso 4: Proyectar los datos

Una vez que tenemos los eigenvectors, proyectamos los datos originales sobre estos nuevos ejes. Cada componente principal es una combinación lineal de las features originales:

$$PC_1 = w_{1,1} \cdot x_1 + w_{1,2} \cdot x_2 + \dots + w_{1,p} \cdot x_p$$

donde $w_{1,j}$ son los pesos (componentes del eigenvector correspondiente).

Paso 5: Seleccionar los K primeros componentes

Finalmente, nos quedamos solo con los primeros K componentes principales (los que tienen mayor varianza). Así reducimos de p features originales a K componentes, con $K \ll p$.

3.2. Ejemplo Simplificado

Supongamos que tenemos datos de estudiantes con dos features: “Horas de Estudio” y “Nota en el Examen”. Estas variables están muy correlacionadas (quien estudia más, saca mejor nota). Si graficamos los datos en 2D, vamos a ver que los puntos están dispersos a lo largo de una diagonal. PCA encuentra esa diagonal como la primera componente principal (la dirección de máxima varianza). La segunda componente principal sería perpendicular a esa diagonal, capturando pequeñas variaciones (ej: estudiantes que estudian mucho pero les fue mal por nervios, o estudiantes que estudiaron poco pero tuvieron suerte). Si queremos reducir a 1 dimensión, nos quedamos solo con PC1 (la diagonal), que captura la mayor parte de la información.

4. Varianza Explicada

Una pregunta clave es: **¿Con cuántos componentes principales nos quedamos?** Para responder esto, miramos la **varianza explicada** por cada componente. Recordá que cada eigenvalue nos dice cuánta varianza captura ese componente.

La proporción de varianza explicada por el componente i es:

$$\text{Varianza Explicada}_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}$$

donde λ_i es el eigenvalue del componente i , y p es el número total de features.

4.1. Regla del 80-90 %

Una regla práctica es quedarse con los componentes que en total expliquen al menos el 80-90 % de la varianza total. Por ejemplo:

- PC1 explica 50 % de la varianza
- PC2 explica 25 %
- PC3 explica 15 %
- PC4 explica 5 %
- PC5+ explican menos del 5 %

En este caso, con 3 componentes (PC1, PC2, PC3) ya cubrís el 90 % de la varianza, entonces podés descartar PC4 en adelante.

4.2. Gráfico de Varianza Acumulada

Una forma visual de decidir es graficar la **varianza acumulada**. Buscás el punto donde la curva se “aplana”, similar al método del codo en K-Means.

5. Implementación en Python

Veamos cómo aplicar PCA en Python usando `scikit-learn`.

5.1. Imports Necesarios

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.decomposition import PCA
5 from sklearn.preprocessing import StandardScaler
```

5.2. Estandarizar y Aplicar PCA

```
1 # Supongamos que X es nuestro DataFrame con las features
2 # Por ejemplo: X = df[['feature1', 'feature2', ..., '
   feature64']]
3
4 # Paso 1: Estandarizar
5 scaler = StandardScaler()
6 X_scaled = scaler.fit_transform(X)
7
8 # Paso 2: Aplicar PCA (por ahora, con todos los componentes)
9 pca = PCA()
10 X_pca = pca.fit_transform(X_scaled)
11
12 # Ver cuanta varianza explica cada componente
13 print("Varianza explicada por cada componente:")
14 print(pca.explained_variance_ratio_)
15
16 # Varianza acumulada
17 print("\nVarianza acumulada:")
18 print(np.cumsum(pca.explained_variance_ratio_))
```

5.3. Reducir a K Componentes

```
1 # Decidimos quedarnos con 2 componentes para visualizacion
2 pca = PCA(n_components=2)
3 X_pca = pca.fit_transform(X_scaled)
4
5 print(f"Forma original: {X_scaled.shape}")
6 print(f"Forma reducida: {X_pca.shape}")
7 print(f"Varianza explicada: {pca.explained_variance_ratio_.
   sum():.2%}")
```

5.4. Visualizar Varianza Explicada

```
1 # Grafico de varianza explicada por componente
2 plt.figure(figsize=(10, 6))
3 plt.bar(range(1, len(pca.explained_variance_ratio_) + 1),
4         pca.explained_variance_ratio_)
5 plt.xlabel('Componente Principal')
6 plt.ylabel('Varianza Explicada')
7 plt.title('Varianza Explicada por Componente')
```

```

8 plt.show()
9
10 # Grafico de varianza acumulada
11 plt.figure(figsize=(10, 6))
12 plt.plot(range(1, len(pca.explained_variance_ratio_) + 1),
13          np.cumsum(pca.explained_variance_ratio_), marker='o',
14          )
15 plt.xlabel('Numero de Componentes')
16 plt.ylabel('Varianza Acumulada')
17 plt.title('Varianza Acumulada')
18 plt.axhline(y=0.9, color='r', linestyle='--', label='90%
19             varianza')
20 plt.legend()
21 plt.grid(True)
22 plt.show()

```

6. Interpretación de Componentes Principales

Los componentes principales son combinaciones lineales de las features originales. Podés ver qué features contribuyen más a cada componente mirando los “loadings” (pesos).

```

1 # Ver los loadings del primer componente principal
2 loadings = pca.components_[0]
3 feature_names = X.columns # Asumiendo que X es un DataFrame
4
5 # Crear DataFrame de loadings
6 loadings_df = pd.DataFrame({
7     'feature': feature_names,
8     'loading': loadings
9 }).sort_values('loading', key=abs, ascending=False)
10
11 print("Contribucion de cada feature a PC1:")
12 print(loadings_df.head(10))

```

Por ejemplo, si estás trabajando con datos de vinos y ves que PC1 tiene loadings altos en “acidez”, “pH”, y “contenido alcohólico”, podés interpretar PC1 como un eje de “composición química”.

7. Reconstrucción de Datos

Una vez que aplicaste PCA y te quedaste con K componentes, podés **reconstruir** una aproximación de los datos originales. Obviamente, si $K < \text{número de features originales}$, va a haber pérdida de información.

```
1 # Reconstruir datos originales desde componentes principales
2 X_reconstructed = pca.inverse_transform(X_pca)
3
4 # Calcular error de reconstruccion (MSE)
5 mse = np.mean((X_scaled - X_reconstructed) ** 2)
6 print(f"Error de reconstruccion (MSE): {mse:.4f}")
```

La reconstrucción es útil para compresión de datos (ej: compresión de imágenes).

8. PCA en el Pipeline de Machine Learning

PCA se puede usar como preprocesamiento antes de entrenar un modelo supervisado. Esto puede:

- Reducir el tiempo de entrenamiento
- Mitigar overfitting (menos features = menos complejidad)
- Eliminar multicolinealidad

8.1. Ejemplo de Pipeline

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.pipeline import Pipeline
4
5 # Dividir datos
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.2, random_state=42
8 )
9
10 # Crear pipeline: Scaler -> PCA -> Modelo
11 pipeline = Pipeline([
12     ('scaler', StandardScaler()),
13     ('pca', PCA(n_components=10)),
```



```

14     ('classifier', LogisticRegression(max_iter=1000))
15 ])
16
17 # Entrenar
18 pipeline.fit(X_train, y_train)
19
20 # Evaluar
21 score = pipeline.score(X_test, y_test)
22 print(f"Accuracy con PCA: {score:.3f}")

```

Podés comparar el performance con y sin PCA para ver si vale la pena.

9. Ventajas y Limitaciones de PCA

9.1. Ventajas

- **Reduce dimensionalidad** sin necesidad de seleccionar manualmente features
- **Elimina multicolinealidad**: los componentes principales son ortogonales (no correlacionados)
- **Visualización**: permite graficar datos de alta dimensión en 2D o 3D
- **Eficiencia computacional**: reduce el costo de entrenar modelos
- **Elimina ruido**: componentes con poca varianza suelen ser ruido

9.2. Limitaciones

- **Pérdida de interpretabilidad**: los componentes principales son combinaciones de features, difíciles de interpretar
- **Asume linealidad**: PCA solo encuentra relaciones lineales. Si las features están relacionadas de forma no lineal, PCA no va a funcionar bien (para eso existe kernel PCA o t-SNE)
- **Sensible a outliers**: los componentes pueden estar distorsionados por valores extremos
- **Requiere estandarización**: sin estandarizar, las features con mayor rango dominan

- **No supervisa:** PCA no considera la variable objetivo (y). Podría eliminar features importantes para la predicción

10. ¿Cuándo Usar PCA?

10.1. Casos de Uso Ideales

Usá PCA cuando:

- Tenés muchas features correlacionadas
- Querés visualizar datos de alta dimensión
- El entrenamiento de modelos es muy lento por la cantidad de features
- Tenés problemas de multicolinealidad (ej: antes de regresión lineal)
- Querés hacer compresión de datos (ej: imágenes, audio)

10.2. Casos Donde NO Usar PCA

No uses PCA si:

- Tus features no están correlacionadas (PCA no va a reducir mucho)
- La interpretabilidad es crítica (los componentes son difíciles de interpretar)
- Las relaciones entre features son no lineales (considerá kernel PCA o autoencoders)
- Tenés muy pocas features (no tiene sentido reducir si ya tenés 3-4 variables)

11. PCA vs Otras Técnicas de Reducción de Dimensionalidad

Existen otras técnicas además de PCA:

- **t-SNE**: mejor para visualización, pero no para reducción computacional (es lento)
- **UMAP**: similar a t-SNE pero más rápido y escalable
- **Feature Selection**: seleccionar features importantes en lugar de crear combinaciones (más interpretable)
- **Autoencoders**: redes neuronales que aprenden representaciones comprimidas (no lineales)

PCA es generalmente la primera opción por su simplicidad y velocidad.

12. Ejemplo Aplicado: PCA en Imágenes

Un caso de uso clásico de PCA es compresión de imágenes. Cada píxel es una feature, y PCA encuentra patrones en cómo varían los píxeles.

12.1. Dataset de Dígitos

El dataset `digits` de `sklearn` tiene imágenes de 8x8 píxeles (64 features). Con PCA, podés reducir a 10-20 componentes sin perder mucha información.

```

1 from sklearn.datasets import load_digits
2
3 # Cargar datos
4 digits = load_digits()
5 X = digits.data # 1797 imagenes x 64 pixels
6 y = digits.target # Digitos 0-9
7
8 # Aplicar PCA
9 pca = PCA(n_components=20)
10 X_pca = pca.fit_transform(X)
11
12 print(f"Forma original: {X.shape}")
13 print(f"Forma reducida: {X_pca.shape}")
14 print(f"Varianza explicada: {pca.explained_variance_ratio_.sum():.2%}")

```

Con solo 20 componentes (de 64 originales), podés capturar más del 90 % de la varianza, lo que significa que las imágenes reconstruidas van a ser muy parecidas a las originales.

13. Recursos Extra

Si querés profundizar más en PCA, te recomiendo:

- **StatQuest: PCA clearly explained:** <https://www.youtube.com/watch?v=FgakZw6K1QQ>
- **3Blue1Brown: Eigenvectors and eigenvalues:** <https://www.youtube.com/watch?v=PFDu9oVAE-g>
- **Documentación sklearn:** <https://scikit-learn.org/stable/modules/decomposition.html#pca>